

Study Report: Designing Data-Intensive Applications - Data Engineer

Written by Gagan Pasupuleti

Family: Data Engineer

Level: Advanced

Chapters: 7

Source: Reference: Designing Data-Intensive Applications - Martin Kleppmann

Official sources and free libraries

- <https://dataintensive.net/>

Summary

Study report for **Designing Data-Intensive Applications** by Martin Kleppmann (Advanced level) mapped to the Data Engineer role. Storage, replication, streaming, and distributed systems.

Chapters

Track: Book-Intro

Chapter 1: About Designing Data-Intensive Applications [Advanced]

Chapter focus: About Designing Data-Intensive Applications** **(Level: Advanced)**

This study report summarizes **Designing Data-Intensive Applications** by Martin Kleppmann for the Data Engineer role. The resource is rated Advanced level. Storage, replication, streaming, and distributed systems. Use this report alongside the official material to prepare for CodeQuest review.

Code Reference:

```
# Designing Data-Intensive Applications
# Author: Martin Kleppmann
# Role: Data Engineer
# Level: Advanced
```

What it shows: Metadata block records the source book and target role family.

Topic guide

What it actually is

A structured study report based on Designing Data-Intensive Applications.

When to use it

When learning Data Engineer skills at Advanced level.

Where to use it

Data Engineer training paths and certification prep.

Real use example

A learner reads this report before starting the full book or free online guide.

Track: Book-Context

Chapter 2: Why This Book Matters for Your Role [Advanced]

Chapter focus: Why This Book Matters for Your Role** *(Level: Advanced)

Data Engineer professionals use ideas from Designing Data-Intensive Applications to solve real workplace problems. Storage, replication, streaming, and distributed systems. This chapter explains how the book fits into your learning path and what you should be able to do after studying it.

Code Reference:

Role: Data Engineer

Book focus: Storage, replication, streaming, and distributed systems.

Recommended level: Advanced

What it shows: Connects the book topic to the job role outcomes.

Topic guide

What it actually is

Role-aligned learning connects theory to job tasks.

When to use it

During career planning and syllabus design.

Where to use it

Data Engineer bootcamps and CodeQuest teacher assignments.

Real use example

A teacher assigns this book report before a advanced skills checkpoint.

Track: Book-Topics

Chapter 3: Key Topics Covered [Advanced]

Chapter focus: Key Topics Covered** *(Level: Advanced)

The main topics in Designing Data-Intensive Applications include practical concepts described as: Storage, replication, streaming, and distributed systems. Study each topic with hands-on practice. Take notes on definitions, workflows, and examples that match tools used in Data Engineer jobs today.

Code Reference:

- Topic 1: core concept
- Topic 2: core concept
- Topic 3: core concept
- Topic 4: core concept

What it shows: Topic list guides what to study chapter-by-chapter in the source.

Topic guide

What it actually is

Topic maps turn a book into actionable learning objectives.

When to use it

Before reading and while building chapter summaries.

Where to use it

Self-study, flipped classroom, and revision.

Real use example

A student maps each book chapter to a CodeQuest report section.

Track: Book-Applied

Chapter 4: Applied: Kafka and Spark Structured Streaming [Advanced]

Chapter focus: Kafka and Spark Structured Streaming** *(Level: Advanced)

Structured Streaming reads Kafka topics with subscribe option; watermarks bound late data; checkpoint locations enable exactly-once-ish sinks. Monitor consumer lag.

Code Reference:

```
df = (spark.readStream.format('kafka')
      .option('subscribe', 'clickstream')
      .load())
query = df.writeStream.format('delta').option('checkpointLocation', 's3://chk/').start()
```

What it shows: checkpointLocation stores offsets - restart without duplicating sinks.

Topic guide

What it actually is

Stream processing handles continuous event pipelines.

When to use it

When sub-minute latency is required for ops or fraud use cases.

Where to use it

Ad-tech, gaming telemetry, and IoT.

Real use example

Consumer lag alert fires at 10k messages - autoscaler adds Spark executors.

Chapter 5: Applied: Data Mesh and Data Contracts [Advanced]

Chapter focus: Data Mesh and Data Contracts *(Level: Advanced)**

Data mesh decentralizes ownership to domains with federated governance. Data contracts define schema, SLAs, and quality between producers/consumers - enforced in CI.

Code Reference:

```
contract:
  name: orders_v2
  schema: { order_id: uuid, amount: decimal }
  sla: { freshness_minutes: 60 }
```

What it shows: YAML contracts versioned in Git gate breaking schema changes.

Topic guide**What it actually is**

Data mesh + contracts scale ownership beyond central platform team.

When to use it

When central data teams become bottlenecks for domain teams.

Where to use it

Large enterprises adopting product-thinking for data.

Real use example

Checkout domain publishes orders_v2 contract - downstream ML features pin version.

Chapter 6: Applied: Pipeline Observability with OpenLineage [Advanced]

Chapter focus: Pipeline Observability with OpenLineage *(Level: Advanced)**

Emit lineage events from Airflow, Spark, and dbt to Marquez/DataHub. Correlate failures to downstream dashboards. SLI: freshness, completeness, error rate.

Code Reference:

```
from openlineage.client import OpenLineageClient
```

```
# emit START/COMPLETE events per task with run_id
```

What it shows: Lineage links Airflow task failures to impacted BI dashboards.

Topic guide

What it actually is

Observability makes data platforms debuggable like microservices.

When to use it

When pipeline failures impact multiple downstream dashboards and teams.

Where to use it

Data platform SRE and incident response.

Real use example

PagerDuty shows which gold table stale - root cause traced to upstream API timeout.

Track: Book-Plan

Chapter 7: Study Plan and Practice [Advanced]

Chapter focus: Study Plan and Practice** *(Level: Advanced)

Finish Designing Data-Intensive Applications with a weekly plan: read one section, write a summary, complete one exercise, and reflect on how it applies to Data Engineer. If a free edition exists, practice every example. Submit your notes and one mini-project demo for teacher review.

Code Reference:

Week 1: Read + notes

Week 2: Exercises

Week 3: Mini project

Week 4: Review + quiz

What it shows: A 4-week plan turns reading into demonstrable skill.

Topic guide

What it actually is

Structured study plans improve retention and portfolio outcomes.

When to use it

After finishing the key topics chapters.

Where to use it

CodeQuest end-of-unit assessments.

Real use example

A learner completes the study plan and uploads a advanced project screenshot.